GUIA TÉCNICO

Neurônio artificial, ativações e perceptron

Capítulo 04/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Derivar um neurônio, comparar ativações e implementar um perceptron em NumPy.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

PROVENIÊNCIA E PRÉ-REQUISITOS

- Material histórico de apoio: Unidade 3 — Redes Neurais Artificiais, páginas 126–181 do PDF consolidado.
-

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

Uma unidade calcula uma combinação linear seguida de uma função de ativação. O perceptron clássico atualiza pesos quando classifica incorretamente um exemplo; só converge sob separabilidade linear.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: $z=w^{\top}x+b$; $\hat{y}=1[z\geq 0]$

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Verifique as hipóteses matemáticas do exemplo, os formatos das entradas e saídas e a relação entre o problema e o algoritmo de treino.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
X=np.array([[0,0],[0,1],[1,0],[1,1]],float)
y=np.array([0,0,0,1]);w=np.zeros(2);b=0.
for epoch in range(15):
 for xi,yi in zip(X,y):
  pred=int(xi@w+b>=0)
  w+=.2*(yi-pred)*xi; b+=.2*(yi-pred)
pred=(X@w+b>=0).astype(int)
print('AND perceptron:',pred.tolist(),'pesos=',w.tolist(),'bias=',b)
assert np.array_equal(pred,y)
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

```
import numpy as np

X = np.array([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
rotulos = {'AND': np.array([0, 0, 0, 1]), 'XOR': np.array([0, 1, 1, 0])}

def treinar_perceptron(X, y, taxa=0.2, epocas=100):
    pesos = np.zeros(X.shape[1], dtype=float)
    vies = 0.0
    historico = []
    for _ in range(epocas):
        erros = 0
        for xi, yi in zip(X, y):
            previsto = int(xi @ pesos + vies >= 0)
            delta = int(yi) - previsto
            pesos += taxa * delta * xi
            vies += taxa * delta
            erros += int(delta != 0)
        historico.append(erros)
    return pesos, vies, historico

for nome, y in rotulos.items():
    w, b, historico = treinar_perceptron(X, y)
    previsto = (X @ w + b >= 0).astype(int)
    acertos = int(np.sum(previsto == y))
    print(nome, 'previsões:', previsto.tolist(), 'acertos:', f'{acertos}/4',
          'erros na última época:', historico[-1])
    if nome == 'AND':
        assert np.array_equal(previsto, y)
    else:
        assert acertos < 4, 'Um único perceptron não deve resolver XOR.'
```

05 EXERCÍCIO E SOLUÇÃO COMENTADA

O que a ativação ReLU devolve quando z = −2?

Resposta: Zero, pois ReLU(z)=max(0,z).

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências,

confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide

REVISÃO DIDÁTICA · CAPÍTULO 04

Perceptron, ativações e XOR

1. Conceito, escopo e interpretação

O perceptron calcula uma combinação afim seguida de ativação. Uma única fronteira linear consegue separar AND, mas não XOR. Uma MLP acrescenta representações intermediárias não lineares que contornam essa limitação geométrica.

2. Formalização e exemplo resolvido

z = w1*x1 + w2*x2 + b; y_hat = ativacao(z). ReLU(z)=max(0,z). Para XOR: (0,0)->0; (0,1)->1; (1,0)->1; (1,1)->0.

3. Relação com o notebook e experimento

Reproduza AND/XOR no notebook e represente os quatro pontos em um gráfico 2D. Verifique se existe uma reta que separe os dois pontos positivos de ambos os negativos.

4. Limitações e erros frequentes

Uma rede de várias camadas sem nenhuma não linearidade continua equivalente a uma transformação afim. Diferencie decisão por degrau, probabilidade por sigmoid e ativação interna ReLU.

5. Exercício comentado

Por que uma saída sigmoide não torna uma rede com apenas uma combinação afim capaz de separar XOR? Porque seu limiar continua gerando uma fronteira linear.

6. Ampliação conceitual do guia

O perceptron linear gera uma fronteira hiperplana e pode separar AND, mas não resolve XOR com uma única fronteira linear. Funções como ReLU e tanh tornam possíveis transformações não lineares em múltiplas camadas.

7. Orientação de estudo e validação

No laboratório do notebook, compare previsões para AND e XOR, usando a mesma codificação de entradas e rótulos. A impossibilidade de separar XOR por reta não prova que a rede jamais aprende: mostra a limitação da hipótese linear.

8. Exercício adicional de fixação

Exercício: desenhe os quatro pontos de XOR no plano cartesiano. Resposta: pontos de classe 1 em (0,1) e (1,0), impossíveis de separar dos demais por uma reta.

Verificação: execute o notebook homônimo no ambiente apropriado; registre versões, shapes, sementes e métricas. Laboratórios Keras opcionais não foram executados durante esta revisão documental.

Abrir notebook do capítulo no repositório